

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY  
SCHOOL OF INFORMATION AND COMMUNICATION  
TECHNOLOGY



---

## FINAL PROJECT REPORT

Topic: **MPI-Based Parallelization of YOLO11n Inference for  
People Counting**

---

Course: **Parallel and Distributed Programming**

Course code: IT4130E

Class/session: 168069 - Friday morning, periods 1–4, C7-215

Lecturer: Nguyen Tuan Dung

Group members: Luu Hieu An - 202400093

Nguyen Phu An - 20235466

Pham Chi Bang - 20235477

Nguyen Thanh Lam - 20235518

Hanoi, 2026

# Contents

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                      | <b>1</b>  |
| <b>1 Introduction</b>                                                | <b>2</b>  |
| <b>2 Problem Description</b>                                         | <b>3</b>  |
| 2.1 Video People Counting as a Parallel Inference Workload . . . . . | 3         |
| 2.2 YOLO11n Inference Pipeline . . . . .                             | 3         |
| 2.3 Frame, Tile, Bounding Box, and Count Definition . . . . .        | 3         |
| 2.4 Serial Baseline and Correctness Target . . . . .                 | 4         |
| <b>3 Parallelization Method</b>                                      | <b>5</b>  |
| 3.1 Level of Parallelism . . . . .                                   | 5         |
| 3.2 Temporal-Spatial Data Decomposition . . . . .                    | 5         |
| 3.3 1D Task Mapping and Static Scheduling . . . . .                  | 6         |
| 3.4 Dynamic Master-Worker Scheduling . . . . .                       | 6         |
| 3.5 Communication Strategy and Star Topology . . . . .               | 6         |
| 3.6 Blocking Communication and MPI Polling . . . . .                 | 7         |
| 3.7 Load Balancing Considerations . . . . .                          | 7         |
| 3.8 Post-processing and NMS Deduplication . . . . .                  | 8         |
| 3.9 Parallel Algorithm Pseudocode . . . . .                          | 8         |
| <b>4 Experimental Setup</b>                                          | <b>9</b>  |
| 4.1 Hardware and Cluster Environment . . . . .                       | 9         |
| 4.2 Software Environment . . . . .                                   | 9         |
| 4.3 Dataset and Input Size Definition . . . . .                      | 9         |
| 4.4 Metrics and Timing Methodology . . . . .                         | 9         |
| 4.5 Required Experiment Matrix . . . . .                             | 10        |
| <b>5 Results and Discussion</b>                                      | <b>11</b> |
| 5.1 Correctness Verification . . . . .                               | 11        |
| 5.2 Input Size Selection . . . . .                                   | 11        |
| 5.3 Granularity and Load Balance . . . . .                           | 12        |
| 5.4 Static vs Dynamic Scheduling . . . . .                           | 12        |
| 5.5 Speedup and Efficiency . . . . .                                 | 12        |
| 5.6 Communication Overhead . . . . .                                 | 13        |

---

|                                   |           |
|-----------------------------------|-----------|
| 5.7 Bottleneck Analysis . . . . . | 13        |
| <b>6 Conclusion</b>               | <b>14</b> |
| <b>7 Member Contributions</b>     | <b>15</b> |

# Abstract

This report presents an MPI-based system for parallel YOLO11n people-counting inference on video. The work is evaluated as a parallel and distributed programming project: YOLO11n is the local inference workload, while the implemented contribution is the C++17/OpenMPI runtime that decomposes video frames into tasks, schedules those tasks across ranks, gathers detections, and writes measurement CSV files.

The implemented method uses temporal-spatial data decomposition. A video is split into frames, each frame is split into tiles, and each frame-tile pair becomes one MPI task. The code supports static block-cyclic scheduling and dynamic master-worker scheduling. Communication is primarily blocking: task and result payloads use `MPI_Send/MPI_Recv`, and static aggregation uses blocking collectives. `MPI_Iprobe` is used only as a polling mechanism in the dynamic scheduler, not as non-blocking payload transfer. No VGG16, convolution-layer parallelization, or halo-exchange method is implemented or reported.

At the time of this draft, no real experiment CSV files were present in the inspected repository snapshot. Therefore, the Results chapter contains clearly marked placeholders for measured CSV evidence and does not fabricate correctness, runtime, speedup, hardware, or communication numbers.

# Chapter 1

## Introduction

Video people counting is a practical workload for studying MPI parallelization. A video naturally provides many independent units of work, but the cost of each unit can vary because crowded frames, tile boundaries, and heterogeneous machines can create load imbalance. This makes the project suitable for analyzing decomposition, mapping, communication topology, scheduling, and speedup.

The project parallelizes inference around YOLO11n rather than modifying YOLO itself. Each MPI rank runs local detector work, while the C++17/OpenMPI program controls task generation, rank scheduling, aggregation, and CSV output. The report therefore focuses on the system behavior of the MPI runtime.

The final direction is YOLO-only. Repository inspection found an implemented frame/tile YOLO-MPI pipeline with static and dynamic schedulers. It did not find an implemented VGG16, convolution, stencil, or halo-exchange method, so those topics are excluded from the report method and evaluation.

The main objectives are:

- define video people counting as a parallel inference workload;
- decompose frames and tiles into independent MPI tasks;
- compare static block-cyclic mapping with dynamic master-worker scheduling;
- describe the actual blocking communication behavior and MPI\_Iprobe polling;
- prepare a results section that can be filled only from real CSV outputs.

## Chapter 2

# Problem Description

### 2.1 Video People Counting as a Parallel Inference Workload

Let the input video be a finite sequence of frames:

$$V = \{F_1, F_2, \dots, F_N\}. \quad (2.1)$$

For each frame, the goal is to count detected people after post-processing:

$$\text{count}(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \quad (2.2)$$

This workload is parallel because different frame regions can be processed independently before final aggregation.

### 2.2 YOLO11n Inference Pipeline

The detector backend is YOLO11n through a Python worker process [1]. The MPI program does not implement neural-network layers; it launches long-lived local detector helpers and sends them frame-tile inference requests. The C++ side owns the parallel runtime: MPI initialization, task generation, scheduling, timing, communication, post-processing, and CSV output.

### 2.3 Frame, Tile, Bounding Box, and Count Definition

For a tile grid with  $R$  rows and  $C$  columns, frame  $F_i$  is decomposed into:

$$F_i \rightarrow \{T_{i,1}, T_{i,2}, \dots, T_{i,R \times C}\}. \quad (2.3)$$

Each tile can be expanded with an overlap margin so that people near tile borders are still visible to the detector. A task is one frame-tile inference:

$$\text{task}(i, j) = T_{i,j}. \quad (2.4)$$

The total task count for  $N$  frames is:

$$|\mathcal{T}| = N \times R \times C. \quad (2.5)$$

A detection is represented as a bounding box plus confidence and class label. Rank 0 converts tile-local detections to frame coordinates, removes duplicates, and writes the final frame count.

## 2.4 Serial Baseline and Correctness Target

The serial baseline processes the same frame-tile task list without MPI distribution and then applies the same post-processing pipeline. Correctness is therefore serial-versus-parallel consistency, not human-label accuracy. For frame counts:

$$\Delta_i = |\text{count}_{\text{parallel}}(F_i) - \text{count}_{\text{serial}}(F_i)|. \quad (2.6)$$

The final correctness table will be filled from measured CSV output generated with `--verify`

1. No correctness value is reported in this draft without that evidence.

## Chapter 3

# Parallelization Method

### 3.1 Level of Parallelism

The implementation exploits task-level parallelism over independent frame-tile inference tasks. It also uses data decomposition because the input video is split temporally into frames and spatially into tiles. The neural-network inference inside YOLO11n is not parallelized by this project; it is treated as local work performed by each active MPI rank.

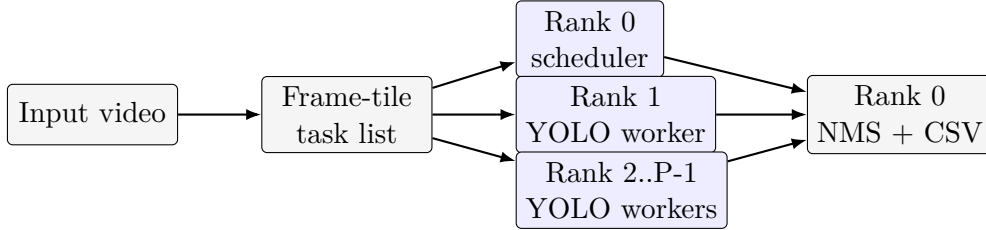


Figure 3.1: System architecture for offline YOLO-MPI people counting.

### 3.2 Temporal-Spatial Data Decomposition

The input is first decomposed across time and then across image space. For  $N$  frames and an  $R \times C$  tile grid, the task set contains:

$$|\mathcal{T}| = N \times R \times C. \quad (3.1)$$

The average task granularity per rank is:

$$G = \frac{N \times R \times C}{P}, \quad (3.2)$$

where  $P$  is the number of MPI ranks. Larger tile grids create more tasks and may improve scheduling flexibility, but they can increase duplicated border work and post-processing overhead.



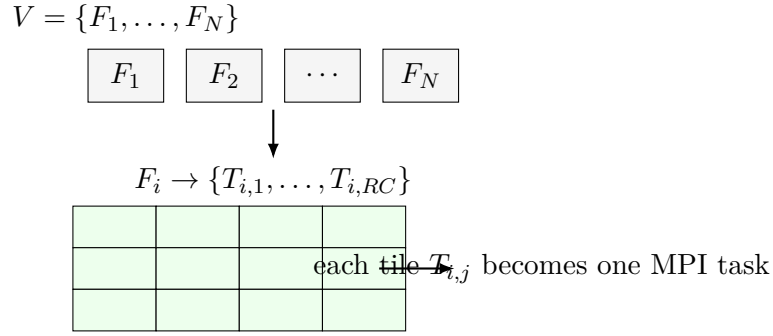


Figure 3.2: Temporal-spatial task decomposition from frames to frame-tile tasks.

### 3.3 1D Task Mapping and Static Scheduling

Static scheduling flattens all tasks into a one-dimensional list. For task index  $i$ , chunk size  $k$ , and  $P$  ranks, the owner is:

$$\text{owner}(\text{task}_i) = \left\lfloor \frac{i}{k} \right\rfloor \bmod P. \quad (3.3)$$

With  $k = 1$ , this becomes round-robin assignment. Static scheduling has low coordination cost because each rank can decide its own tasks locally, but it can suffer when expensive tiles are unevenly distributed.

### 3.4 Dynamic Master-Worker Scheduling

Dynamic scheduling uses rank 0 as a master. Rank 0 owns the remaining task queue, dispatches work to available ranks, receives completed result payloads, and sends the next task. If `master_compute` is enabled, rank 0 may also process local tasks while waiting for worker results.

### 3.5 Communication Strategy and Star Topology

The dynamic scheduler uses a star topology: workers communicate with rank 0, not directly with each other. In static scheduling, ranks work independently and communicate mainly during final aggregation.

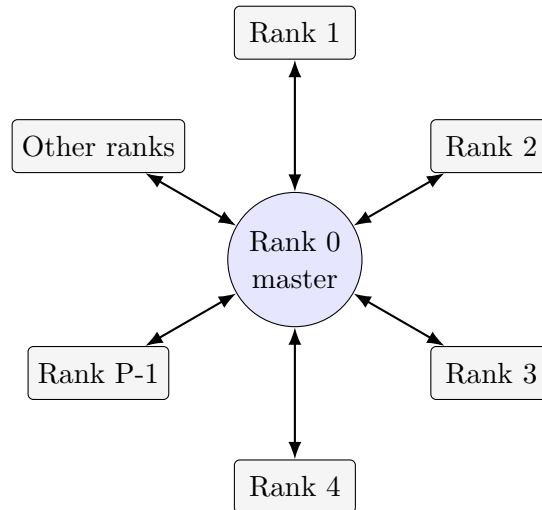


Figure 3.3: Master-worker star communication topology.

### 3.6 Blocking Communication and MPI Polling

The implementation uses blocking message transfer for task and result payloads. Task dispatch uses `MPI_Send`; task reception and result reception use `MPI_Recv`; static aggregation uses `MPI_Gather` and `MPI_Gatherv`. The dynamic scheduler also calls `MPI_Iprobe` so rank 0 can poll whether a worker result is ready before choosing whether to compute locally. This is polling only: the actual payload transfer is still blocking, and no `MPI_Isend` or `MPI_Irecv` payload path is reported.

### 3.7 Load Balancing Considerations

Per-rank time is interpreted as compute time, communication time, and idle/wait time:

$$T_r = T_{compute,r} + T_{comm,r} + T_{idle,r}. \quad (3.4)$$

The imbalance metric used in the report is:

$$imbalance = \frac{T_{max} - T_{min}}{T_{max}}. \quad (3.5)$$

Dynamic scheduling should help when task costs vary, while static scheduling should have lower scheduling overhead. The final conclusion must be based on measured CSV files rather than expectation.

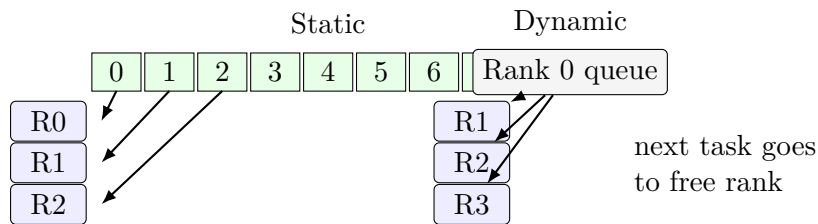


Figure 3.4: Static assignment versus dynamic task dispatch.

### 3.8 Post-processing and NMS Deduplication

After inference, detections are converted from tile coordinates to frame coordinates. Rank 0 applies tile ownership filtering, confidence sorting, global non-maximum suppression, and duplicate rules for boxes near tile boundaries. The final frame count is the number of remaining person boxes.

### 3.9 Parallel Algorithm Pseudocode

---

**Algorithm 1** Serial YOLO people-counting baseline

---

- 1: Build task list  $\mathcal{T}$  from  $N$  frames and  $R \times C$  tiles.
  - 2: **for** each task  $T_{i,j} \in \mathcal{T}$  **do**
  - 3:   Run local YOLO inference on tile  $T_{i,j}$ .
  - 4:   Convert detections to frame coordinates.
  - 5: **end for**
  - 6: Apply tile ownership filtering and global NMS per frame.
  - 7: Write `frame_counts.csv`, `bboxes.csv`, and `summary.csv`.
- 

---

**Algorithm 2** Static MPI YOLO inference

---

- 1: All ranks build the same task list  $\mathcal{T}$ .
  - 2: **for** each task index  $i$  **do**
  - 3:   **if**  $owner(task_i) = \lfloor i/k \rfloor \bmod P$  equals local rank **then**
  - 4:     Run YOLO inference and store local detections and timing.
  - 5:   **end if**
  - 6: **end for**
  - 7: Gather serialized detections and rank metrics to rank 0.
  - 8: Rank 0 applies post-processing and writes CSV outputs.
- 

---

**Algorithm 3** Dynamic master-worker MPI YOLO inference

---

- 1: **if** rank is 0 **then**
  - 2:   Initialize task queue and dispatch initial tasks to workers.
  - 3:   **while** unfinished tasks or active workers remain **do**
  - 4:     Poll for ready worker results using `MPI_Iprobe`.
  - 5:     Receive completed payloads with blocking `MPI_Recv`.
  - 6:     Send next task or termination message with `MPI_Send`.
  - 7:     Optionally compute one local task if `master_compute` is enabled.
  - 8:   **end while**
  - 9:   Post-process all detections and write CSV outputs.
  - 10: **else**
  - 11:   **while** a task is received **do**
  - 12:     Run YOLO inference and send result payload to rank 0.
  - 13:   **end while**
  - 14: **end if**
-

## Chapter 4

# Experimental Setup

### 4.1 Hardware and Cluster Environment

The target environment is the macOS/OpenMPI cluster described by the repository hostfiles. GPU-oriented experiments use `configs/hosts_macos_gpu`; CPU/core-count experiments use `configs/hosts_macos_core` and weighted hostfiles when process counts exceed the three-rank GPU layout. Exact CPU, memory, process-slot, network, OpenMPI, and Python details must be taken from the final hardware evidence logs. They are intentionally not invented in this draft.

Table 4.1: Hardware evidence to be filled from cluster logs

| Host   | Role        | CPU/SoC     | Memory      | Evidence source     |
|--------|-------------|-------------|-------------|---------------------|
| master | Pending log | Pending log | Pending log | final evidence logs |
| node1  | Pending log | Pending log | Pending log | final evidence logs |
| node2  | Pending log | Pending log | Pending log | final evidence logs |

### 4.2 Software Environment

The implementation uses C++17, OpenMPI, a Python virtual environment, Ultralytics YOLO11n, and helper scripts under `scripts/runtime/`. The final report must record the actual OpenMPI version, Python version, YOLO package version, model path, detector device, and source video path from the run logs.

### 4.3 Dataset and Input Size Definition

The input size is the number of processed video frames  $N$ . The input-size selection experiment runs several candidate frame counts and chooses an  $N$  whose wall-clock runtime is approximately 120–180 seconds on the target configuration. Speedup experiments then use  $2N$  frames to reduce startup noise.

### 4.4 Metrics and Timing Methodology

The report uses the CSV fields generated by the implementation:

- `total_ms_with_comm`: wall-clock runtime including MPI communication;
- `total_ms_without_comm`: compute-oriented estimate excluding communication;

- `compute_ms`, `yolo_ms`, `comm_ms`, `idle_ms`: rank-level timing;
- `tasks_done`: task count per rank;
- `correctness_pass`: serial-versus-parallel verification status when `--verify 1`.

Speedup and efficiency are:

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}. \quad (4.1)$$

Communication overhead is:

$$\frac{T_{comm}}{T_{wall}}. \quad (4.2)$$

## 4.5 Required Experiment Matrix

Table 4.2: Required YOLO-only experiment matrix

| Experiment    | Main setting                                   | Required output               | Report purpose                                  |
|---------------|------------------------------------------------|-------------------------------|-------------------------------------------------|
| Correctness   | Static and dynamic,<br><code>--verify 1</code> | correctness CSVs              | Verify parallel output against serial baseline. |
| Input size    | Multiple $N$ ,<br>$P = 12$ , dynamic           | <code>raw/find_N.csv</code>   | Choose $N$ near 120–180 s.                      |
| Granularity   | Selected $N$ , one or more tile grids          | <code>rank_metrics.csv</code> | Per-rank task, compute, comm, idle analysis.    |
| Scheduling    | Same $N, P$ , static vs dynamic                | scheduler CSV                 | Compare runtime and imbalance.                  |
| Speedup       | $2N$ , $P = 1, 2, 4, 8, 12, 24$ if possible    | <code>raw/speedup.csv</code>  | Compute $S_p$ and $E_p$ .                       |
| Communication | Derived from summaries and rank metrics        | communication CSV             | Quantify $T_{comm}/T_{wall}$ .                  |

# Chapter 5

## Results and Discussion

No completed experiment CSV files were present in the inspected repository snapshot. This chapter is a results template for the final measured run package. It must be filled only from real CSV outputs; no runtime, speedup, correctness, hardware, or communication values are inferred.

### 5.1 Correctness Verification

Correctness is evaluated by running the implementation with `--verify 1`. The expected evidence is `summary.csv`, `frame_counts.csv`, `bboxes.csv`, and `rank_metrics.csv` for both static and dynamic scheduling.

Table 5.1: Correctness verification table to be filled from measured CSV

| Schedule | Frames         | Grid           | Processes      | Correctness<br>pass | Evidence        |
|----------|----------------|----------------|----------------|---------------------|-----------------|
| Static   | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV      | static summary  |
| Dynamic  | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV      | dynamic summary |

### 5.2 Input Size Selection

The input-size sweep chooses  $N$  so that wall-clock runtime is approximately 120–180 seconds on the selected cluster configuration.

Table 5.2: Input size selection table to be filled from `find_N.csv`

| $N$            | Grid    | $P$     | Runtime with<br>comm. | Runtime<br>without comm. | Selection note |
|----------------|---------|---------|-----------------------|--------------------------|----------------|
| Pending<br>CSV | Pending | Pending | Pending CSV           | Pending CSV              | Pending CSV    |

### 5.3 Granularity and Load Balance

For each selected granularity case, the report will compute:

$$G = \frac{N \times R \times C}{P}, \quad imbalance = \frac{T_{max} - T_{min}}{T_{max}}. \quad (5.1)$$

Table 5.3: Per-rank granularity and load-balance table to be filled from `rank_metrics.csv`

| Rank    | Tasks          | Compute<br>ms  | YOLO ms        | Comm.<br>ms    | Idle ms        | Host        |
|---------|----------------|----------------|----------------|----------------|----------------|-------------|
| Pending | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | Pending CSV |

### 5.4 Static vs Dynamic Scheduling

Static and dynamic scheduling must be compared using the same  $N$ ,  $P$ , tile grid, detector, model, and source video.

Table 5.4: Static versus dynamic scheduling table to be filled from `scheduler_comparison.csv`

| Schedule | $P$     | Grid    | Runtime<br>with comm. | Runtime<br>without<br>comm. | Imbalance      | Note        |
|----------|---------|---------|-----------------------|-----------------------------|----------------|-------------|
| Static   | Pending | Pending | Pending CSV           | Pending CSV                 | Pending<br>CSV | Pending CSV |
| Dynamic  | Pending | Pending | Pending CSV           | Pending CSV                 | Pending<br>CSV | Pending CSV |

### 5.5 Speedup and Efficiency

Speedup and efficiency are computed from the measured runtime:

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}. \quad (5.2)$$

The final speedup experiment should use input size  $2N$ . If  $P = 24$  exceeds the intended physical-core configuration, it must be labeled as oversubscription.

Table 5.5: Speedup table to be filled from `speedup.csv`

| $P$     | Runtime<br>with comm. | Runtime<br>without<br>comm. | $S_p$          | $E_p$          | Note        |
|---------|-----------------------|-----------------------------|----------------|----------------|-------------|
| Pending | Pending<br>CSV        | Pending<br>CSV              | Pending<br>CSV | Pending<br>CSV | Pending CSV |

## 5.6 Communication Overhead

Communication overhead is:

$$\frac{T_{comm}}{T_{wall}}. \quad (5.3)$$

The numerator must come from `comm_ms_total` or aggregated `rank_metrics.csv`; the denominator must come from `total_ms_with_comm`.

Table 5.6: Communication overhead table to be filled from measured summaries

| Run            | $P$     | $T_{wall}$     | $T_{comm}$     | Ratio          | Evidence                 |      |
|----------------|---------|----------------|----------------|----------------|--------------------------|------|
| Pending<br>CSV | Pending | Pending<br>CSV | Pending<br>CSV | Pending<br>CSV | communication<br>summary | sum- |

## 5.7 Bottleneck Analysis

The final bottleneck analysis should be evidence-driven. Possible causes include task imbalance, heterogeneous host speeds, repeated tile-border work, communication overhead from dynamic scheduling, and Apple MPS contention when multiple ranks share accelerator resources. This draft does not rank those causes because the required CSV evidence has not yet been collected.



## Chapter 6

# Conclusion

This report presents the implemented YOLO-MPI people-counting system as a parallel inference runtime. The method decomposes a video into frame-tile tasks, supports static block-cyclic scheduling and dynamic master-worker scheduling, uses rank-local YOLO workers, and aggregates final detections on rank 0 with duplicate removal and NMS.

The implementation evidence supports a YOLO-only report. A second VGG16, convolution, or halo-exchange method is not implemented and is therefore not included. The communication design is also reported conservatively: payload transfer uses blocking MPI calls, while `MPI_Iprobe` is used only for polling in the dynamic scheduler.

The remaining work before final submission is experimental rather than methodological. The final PDF should replace all pending result placeholders with measured CSV evidence for correctness, input size selection, granularity, scheduling, speedup, and communication overhead.

## Chapter 7

# Member Contributions

Table 7.1 records the draft responsibility split for the YOLO-only report and final evidence package. The group should update it only with confirmed work before submission.

Table 7.1: Draft member contributions

| Member           | Student ID | Contribution                                                                        |
|------------------|------------|-------------------------------------------------------------------------------------|
| Luu Hieu An      | 202400093  | Report integration, repository evidence audit, and final presentation alignment.    |
| Nguyen Phu An    | 20235466   | Cluster setup, OpenMPI execution, and hardware evidence collection.                 |
| Pham Chi Bang    | 20235477   | C++/MPI implementation, scheduling scripts, and result aggregation support.         |
| Nguyen Thanh Lam | 20235518   | Plot generation, speedup table preparation, and discussion of measured limitations. |

# Bibliography

- [1] Ultralytics. Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026.